



S01

Construcción de un servicio base para un sistema distribuido

Microservicio de órdenes - Spring Boot - PostgreSQL - Flyway - OpenAPI

ESTUDIANTE

Aldo Calla Ticona

EQUIPO

Sin grupo

SERVICIO

pagatu-orden-ms

ENTREGA

S01 - 27/08/2026

Informe regenerado con 10 capturas originales completas, sin recorte ni reconstrucción.

00 Resumen de la implementación

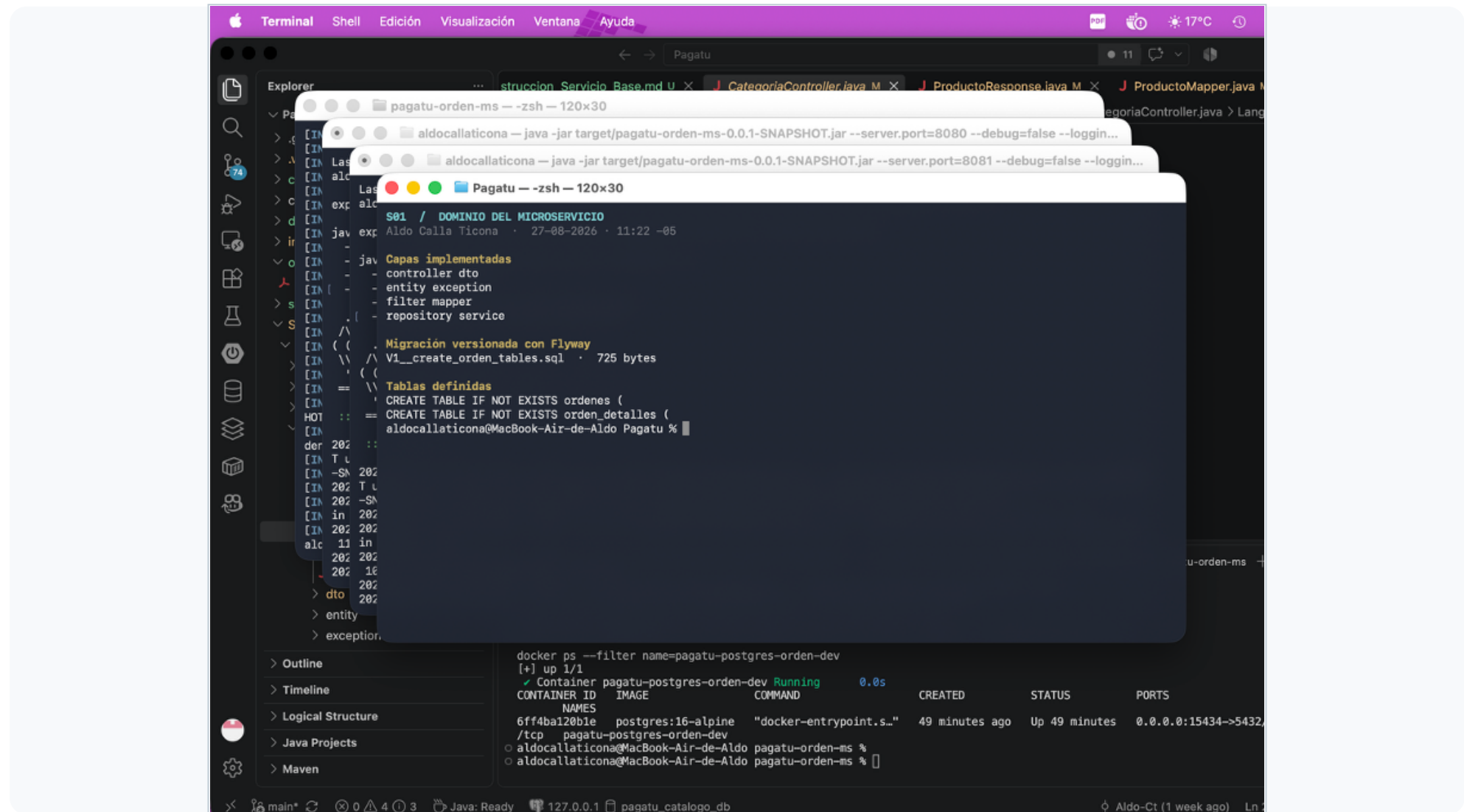
pagatu-orden-ms delimita el dominio de órdenes, persiste datos en PostgreSQL mediante migraciones Flyway, expone un CRUD REST documentado con OpenAPI y puede ejecutarse como dos instancias stateless en puertos distintos.

Criterio	Implementación	Evidencia
Dominio	Capas controller, service, repository, entity, dto, mapper, exception y filter.	Evidencia 01
Persistencia	PostgreSQL 16, migración V1 y registros persistidos.	Evidencias 02-04
API REST	CRUD, validación HTTP 400, recurso HTTP 404 y OpenAPI.	Evidencia 05
Escalamiento	Instancias independientes en 8080 y 8081 con PostgreSQL compartida.	Evidencias 06-10
Reproducibilidad	Java 21, Maven Wrapper, Docker Compose y Actuator.	Documentación

ALCANCE VERIFICADO

Las capturas muestran el ambiente real de trabajo, las herramientas, la base de datos, los endpoints y ambos procesos Java. Cada imagen se incorpora completa y conserva su relación de aspecto original.

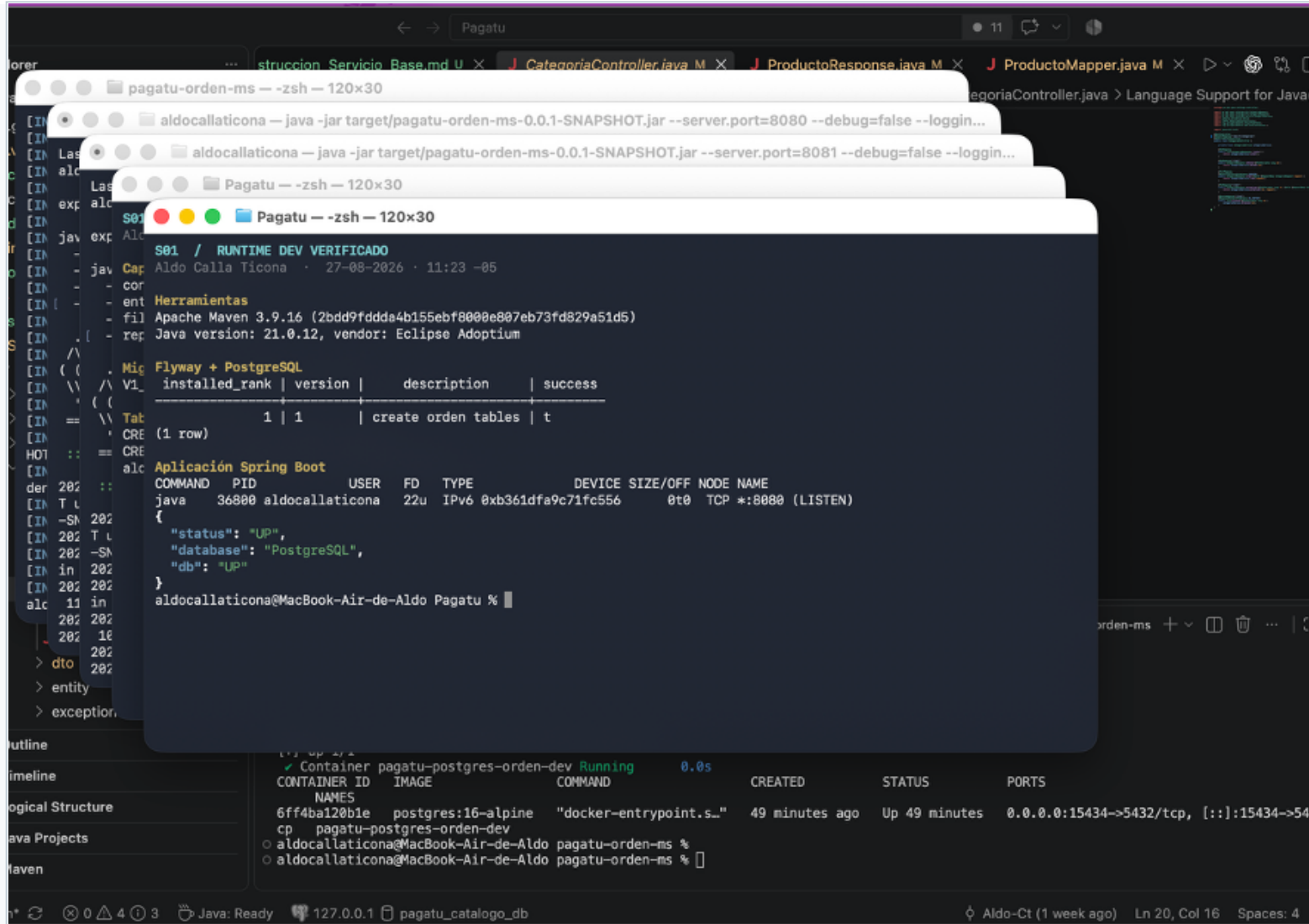
01 Dominio y arquitectura por capas



Evidencia 01. Estructura implementada para el dominio Orden y migración versionada con Flyway.

Original 884 x 678 px - imagen completa - sin recorte

02 Runtime DEV verificado



```

Pagatu
11
Pagatu -- zsh -- 120x30
aldocallaticona -- java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8080 --debug=false --login...
aldocallaticona -- java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8081 --debug=false --login...
Pagatu -- zsh -- 120x30
S01 / RUNTIME DEV VERIFICADO
Aldo Calla Ticona · 27-08-2026 · 11:23 -05
Herramientas
Apache Maven 3.9.16 (2bdd9fddda4b155ebf800e007eb73fd829a51d5)
Java version: 21.0.12, vendor: Eclipse Adoptium
Flyway + PostgreSQL
installed_rank | version | description | success
-----
1 | 1 | create orden tables | t
(1 row)
Aplicación Spring Boot
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
java 36800 aldocallaticona 22u IPv6 0xb361dfa9c71fc556 0t0 TCP *:8080 (LISTEN)
{"status": "UP",
"database": "PostgreSQL",
"db": "UP"}
aldocallaticona@MacBook-Air-de-Aldo Pagatu %

```

Container pagatu-postgres-orden-dev Running 0.0s

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
6ff4ba120b1e	postgres:16-alpine	"docker-entrypoint.s..."	49 minutes ago	Up 49 minutes	0.0.0.0:15434->5432/tcp, [::]:15434->5432/tcp

cp pagatu-postgres-orden-dev

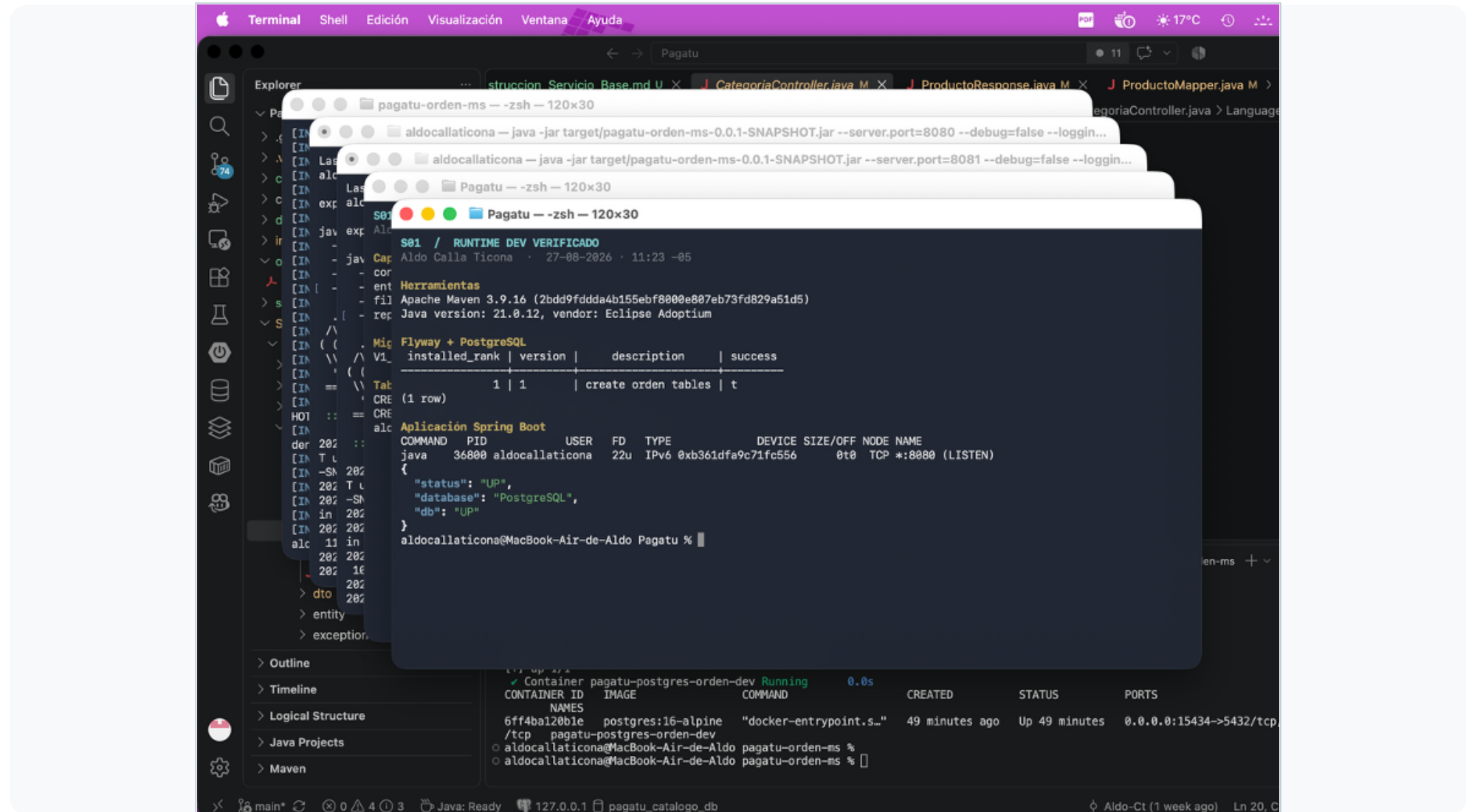
aldocallaticona@MacBook-Air-de-Aldo pagatu-orden-ms %

aldocallaticona@MacBook-Air-de-Aldo pagatu-orden-ms %

Evidencia 02. Java 21, Maven Wrapper, Flyway, proceso Spring Boot y estado de la base de datos.

Original 884 x 624 px - imagen completa - sin recorte

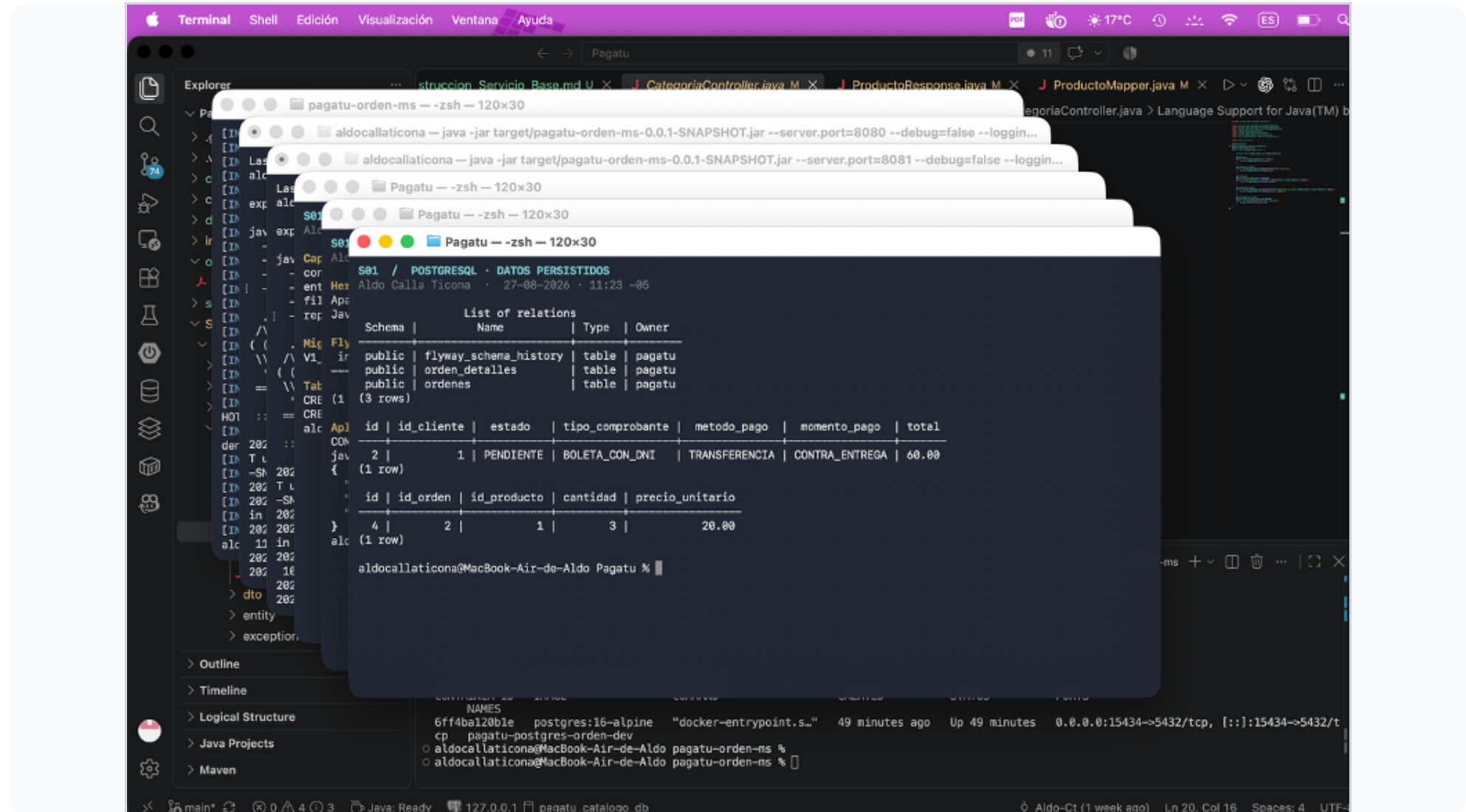
03 Runtime completo en el entorno de trabajo



Evidencia 03. Vista completa de VS Code y la terminal con la validación del ambiente DEV.

Original 884 x 662 px - imagen completa - sin recorte

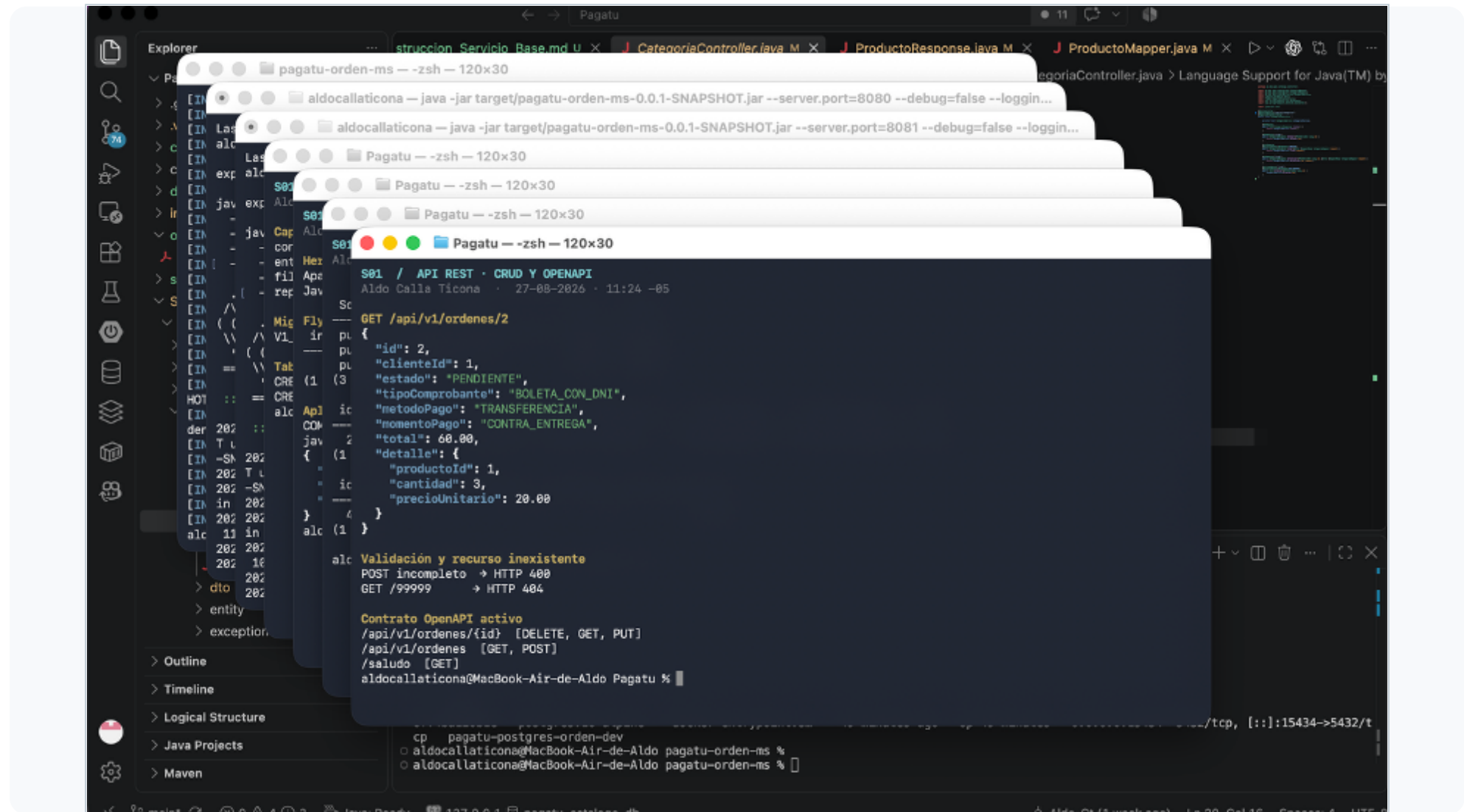
04 Persistencia en PostgreSQL



Evidencia 04. Tablas creadas por Flyway y registros persistidos en ordenes y orden_detalles.

Original 884 x 586 px - imagen completa - sin recorte

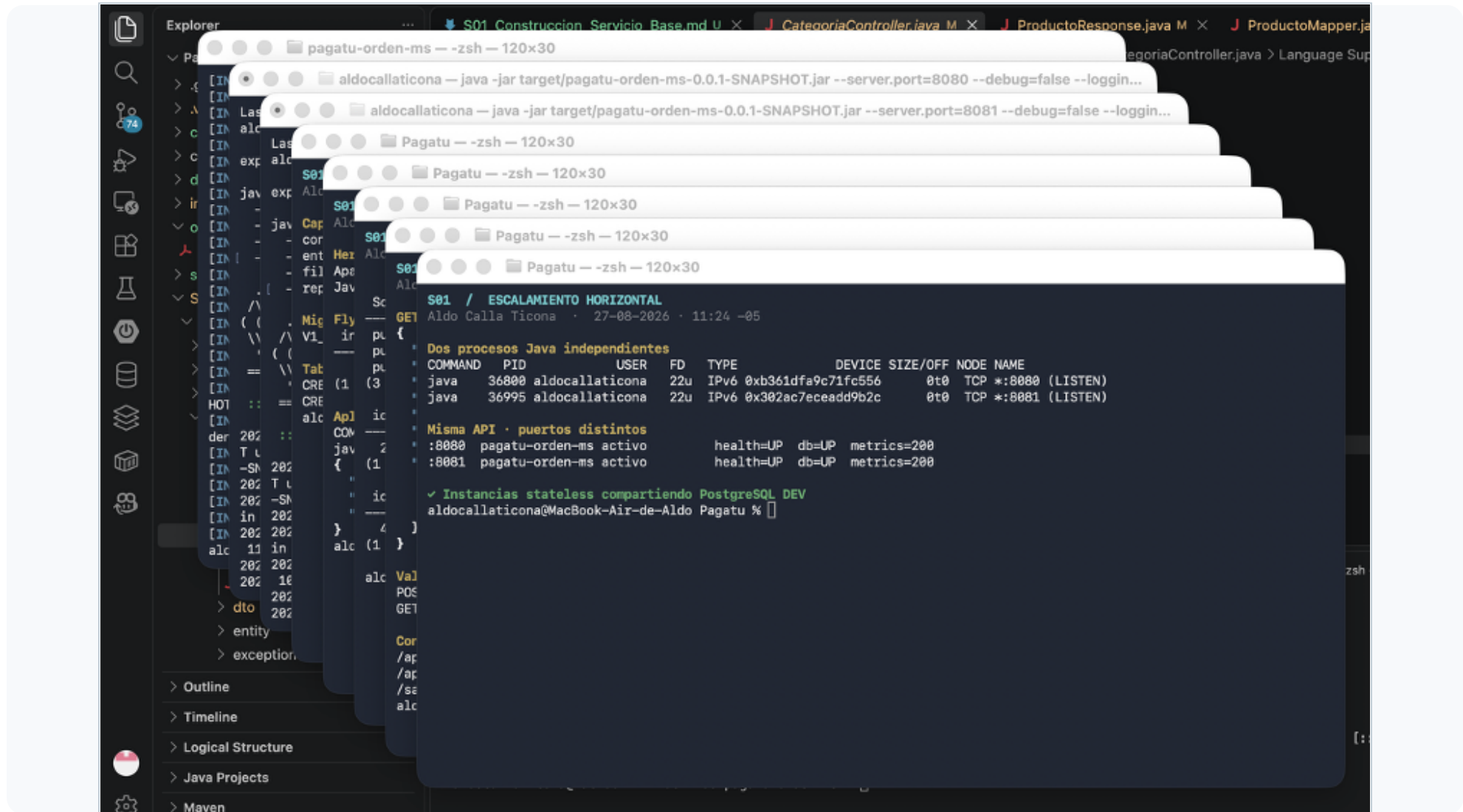
05 CRUD y contrato OpenAPI



Evidencia 05. Respuesta de la API, validación HTTP 400, recurso HTTP 404 y rutas publicadas.

Original 884 x 550 px - imagen completa - sin recorte

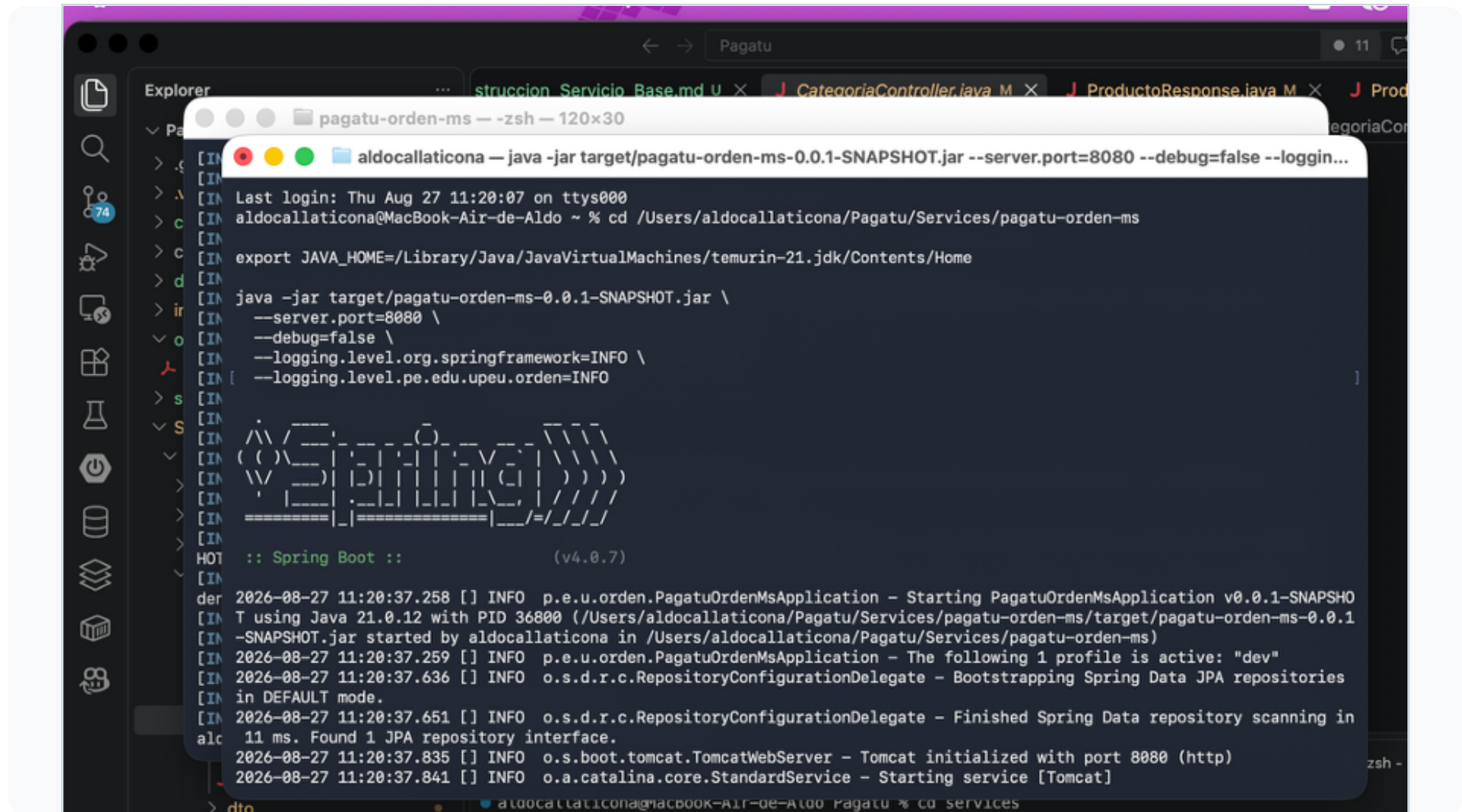
06 Escalamiento horizontal



Evidencia 06. Dos procesos Java independientes en 8080 y 8081 compartiendo PostgreSQL DEV.

Original 884 x 564 px - imagen completa - sin recorte

07 Arranque de la instancia 8080



Evidencia 07. Spring Boot inicia la primera instancia y Tomcat queda asociado al puerto 8080.

Original 884 x 532 px - imagen completa - sin recorte

08 Arranque de la instancia 8081

```

aldocallaticona — java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8081 --debug=false --login...

Last login: Thu Aug 27 11:20:34 on ttys002
aldocallaticona@MacBook-Air-de-Aldo ~ % cd /Users/aldocallaticona/Pagatu/Services/pagatu-orden-ms
export JAVA_HOME=/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home
java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar \
  --server.port=8081 \
  --debug=false \
  --logging.level.org.springframework=INFO \
  --logging.level.pe.edu.upeu.orden=INFO

:: Spring Boot :: (v4.0.7)

2026-08-27 11:21:44.984 [] INFO p.e.u.orden.PagatuOrdenMsApplication - Starting PagatuOrdenMsApplication v0.0.1-SNAPSHO
2026-08-27 11:21:44.985 [] INFO p.e.u.orden.PagatuOrdenMsApplication - The following 1 profile is active: "dev"
2026-08-27 11:21:45.298 [] INFO o.s.d.r.c.RepositoryConfigurationDelegate - Bootstrapping Spring Data JPA repositories
2026-08-27 11:21:45.311 [] INFO o.s.d.r.c.RepositoryConfigurationDelegate - Finished Spring Data repository scanning in
2026-08-27 11:21:45.481 [] INFO o.s.boot.tomcat.TomcatWebServer - Tomcat initialized with port 8081 (http)
2026-08-27 11:21:45.487 [] INFO o.a.catalina.core.StandardService - Starting service [Tomcat]

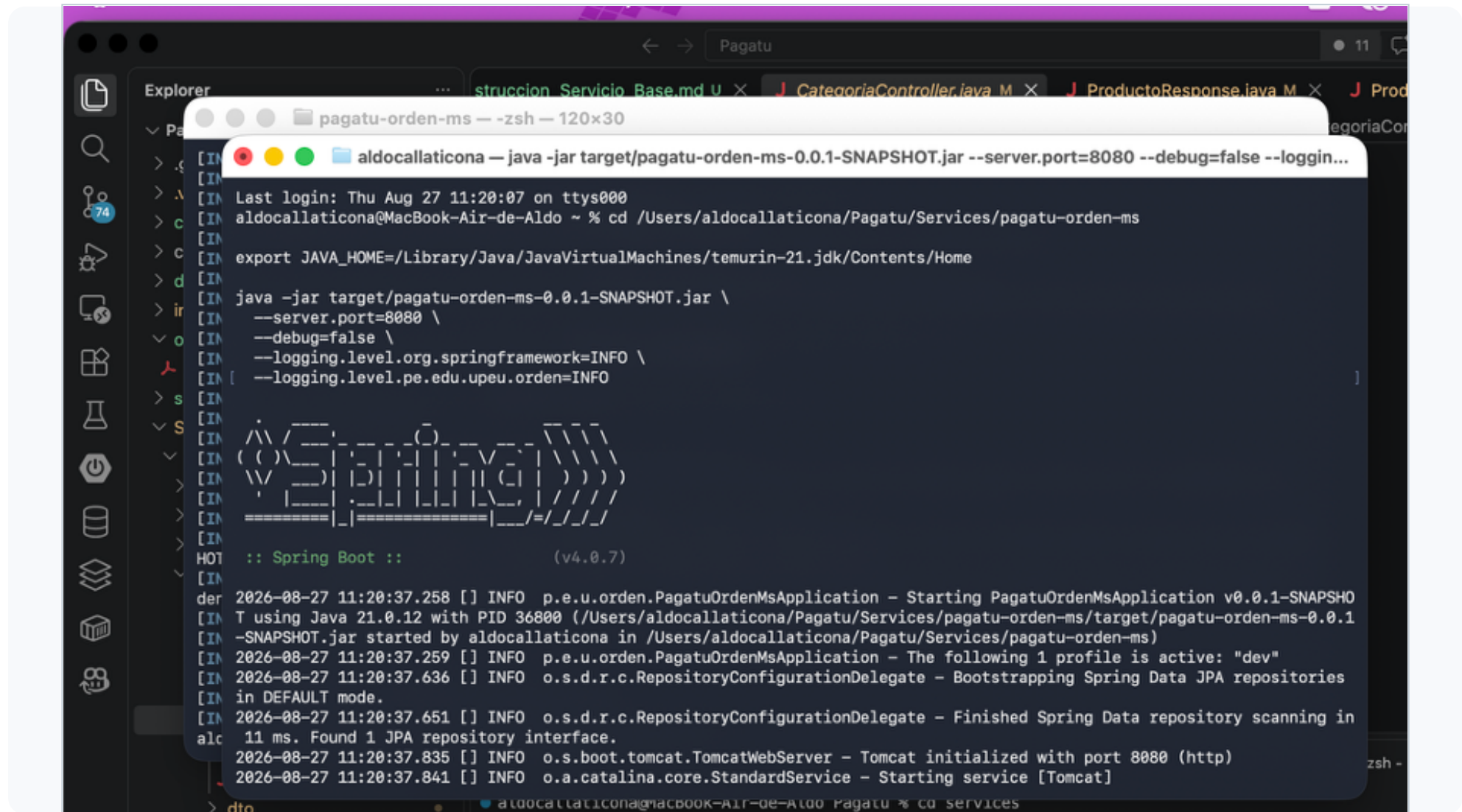
aldocallaticona@MacBook-Air-de-Aldo:~/Services/pagatu-orden-ms$ docker compose -f compose-dev.yml up -d
docker ps --filter name=pagatu-postgres-orden-dev
[+] up 1/1
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS
6ff4ba120b1e   postgres:16-alpine "docker-entrypoint.s..." 49 minutes ago Up 49 minutes 0.0.0.0:154
/tcp pagatu-postgres-orden-dev
aldocallaticona@MacBook-Air-de-Aldo:~/Services/pagatu-orden-ms$
aldocallaticona@MacBook-Air-de-Aldo:~/Services/pagatu-orden-ms$

```

Evidencia 08. El mismo artefacto se ejecuta como segunda instancia mediante server.port=8081.

Original 884 x 720 px - imagen completa - sin recorte

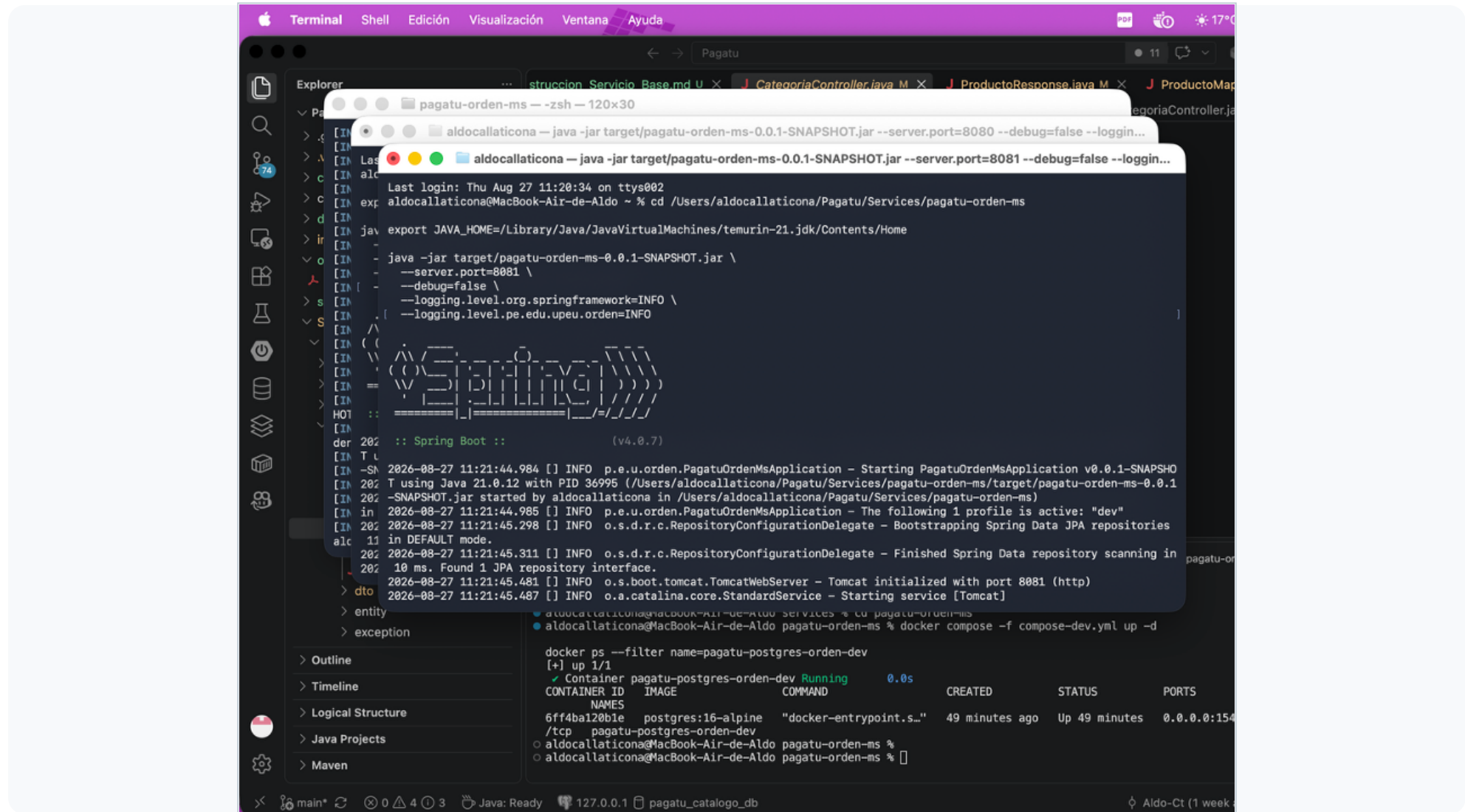
09 Evidencia complementaria de 8080



Evidencia 09. Vista original adicional del arranque de la primera instancia, conservada sin cambios.

Original 884 x 532 px - imagen completa - sin recorte

10 Evidencia complementaria de 8081



Evidencia 10. Vista original adicional del arranque de la segunda instancia, conservada sin cambios.

Original 884 x 720 px - imagen completa - sin recorte

11 Documentación técnica y reproducción

El ambiente DEV se reproduce con Docker Compose y Maven Wrapper. Los puertos se externalizan para evitar acoplamiento y permitir copias equivalentes del servicio.

```
cd Services/pagatu-orden-ms
docker compose -f compose-dev.yml up -d
./mvnw clean package -DskipTests
java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8080
java -jar target/pagatu-orden-ms-0.0.1-SNAPSHOT.jar --server.port=8081
```

Verificación	URL / comando	Resultado
Health 8080	http://localhost:8080/actuator/health	UP - DB UP
Health 8081	http://localhost:8081/actuator/health	UP - DB UP
Swagger	http://localhost:8080/swagger-ui/index.html	HTTP 200
Órdenes	http://localhost:8080/api/v1/ordenes	HTTP 200

HALLAZGO TÉCNICO

El primer fallo de la segunda instancia no correspondía al puerto 8081: Spring no encontraba OrdenMapper durante una compilación concurrente. Un clean package produjo un JAR estable y permitió ejecutar 8080 y 8081 de forma independiente.

12 Reflexión técnica y conclusión

Principio	Aplicación en Pagatu
Reproducibilidad	Maven Wrapper y Docker Compose eliminan configuraciones manuales ocultas.
Persistencia	Flyway versiona el esquema y PostgreSQL conserva los datos fuera del proceso Java.
Desacoplamiento	Orden conserva identificadores de recursos pertenecientes a otros dominios.
Escalabilidad	El puerto se externaliza y las instancias permanecen stateless.
Observabilidad	Actuator verifica salud, base de datos y métricas.

CONCLUSIÓN

pagatu-orden-ms cumple el servicio base de S01: dominio delimitado, persistencia versionada, CRUD documentado, manejo de errores, observabilidad y ejecución horizontal comprobada. La solución queda preparada para integrarse con Config Server, Service Discovery, API Gateway y balanceo de carga.

Repositorio: <https://github.com/Aldo-Ct/Pagalotu>

Archivos originales: [capturas/originales_s01/](#)

13 Anexo - Feedback de la sesión

1. Aprendizaje más importante

Los microservicios permiten dividir responsabilidades y evitar que todo el sistema dependa de un único componente.

2. Punto más confuso

La instalación de dependencias y la creación del proyecto en VS Code y Spring Initializr.

3. Pregunta para la siguiente clase

¿Cuál es la forma recomendada de gestionar dependencias de Spring Boot desde VS Code para evitar incompatibilidades?

4. Nivel de comprensión

Más o menos. Entendí la idea general, pero todavía tengo dudas.

5. Apoyo solicitado

Repasar paso a paso la creación del proyecto y la selección o instalación de dependencias en VS Code.

6. Autoevaluación

Comprometido. Sé que podría haberme esforzado un poco más.

7. Satisfacción

7 de 10.